call to the shop succeeds afterwards[10], and we introduce a discrepancy, possibly even causing the customer to buy the same item twice. See figure 11.7 illustrating this case with sequence diagrams.

If the shops' servers have a timeout set to 8 seconds[11] we need to complete all the remaining operations within 2 seconds, which leaves likely less than a second to run the nearest neighbor search.

In short, when we move to distributed architectures, there are lot more factors we need to be careful about, that go beyond the mere use of algorithms. Nevertheless, the choice of the search algorithm is even more important: in particular, a bad choice can have dire consequences on our web application.

That means we need to be careful about the algorithm we implement:

- Different data structures have different performance for average and worst case: you can decide to go with the fastest algorithm on average, to serve as many requests as possible in the shortest time, or to go with the best for the worst case performance, to be sure that all requests will complete within the allotted time (even if, on average, it will be slower).
- If the dataset to search though constantly grows, past a certain point it will probably become too large to allow you to run a NN search in the time available. At that point, you need to think about other way to scale your application, like for example a geographical sharding of the data, or – if that doesn't make sense for your